EvalForge: Multi-LLM Evaluation and Regression Testing

Oct 10, 2026 · @Vedansh Airen

1. Summary and goals

EvalForge is a web application that runs one evaluation dataset against several large language models (LLMs). It compares them on answer quality, latency, token usage and estimated cost. It also checks whether a change, such as a new system prompt, made a model worse than a baseline, and it gives that verdict only when the comparison is fair.

It was built for a question campus AI teams face again and again: which model answers our users correctly, how fast, at what cost, and did a prompt change help? The design goals were:

- The same inputs for every model.
- Quality, speed and cost reported side by side rather than blended into one score.
- Missing data shown as *unavailable*, never as zero, and every average labeled with the number of cases it covers.
- API keys kept on the server, and spending capped.
- One command to run it locally, in Docker or on Kubernetes.

EvalForge was tested live on two datasets:

- A customer-support dataset.
- A 12-case *Campus AI Enablement Desk* dataset covering Claude for Education, ChatGPT Edu, Gravyty bots, tool pilots and data policy.

On the campus dataset, Claude Haiku 4.5, Claude Sonnet 5 and Gemini 3.8 Flash all averaged about 94 out of 100 for correctness. The Claude models answered about twice as fast. The regression check correctly found no regression for Gemini between two runs.

2. Technical stack

Layer	Technology	Role
API	Python 3.11, FastAPI, Uvicorn	REST endpoints for experiments, runs, results and comparison
Validation and configuration	Pydantic v2, pydantic-settings	Dataset and request validation, judge-output validation, settings from environment variables with keys held as secrets
Storage	SQLAlchemy 2, SQLite	Experiments, case snapshots, model runs and per-case results
Model access	`anthropic`, `google-genai`, `openai` SDKs	One thin adapter per provider behind a shared async interface
Execution	asyncio, FastAPI background tasks	Concurrent calls limited by a semaphore, with retries and timeouts
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS v4, Recharts	Dashboard, create form, results, failure inspector, quality-versus-cost chart
Packaging	Docker (multi-stage builds), Docker Compose	Backend and frontend images, a `bench` profile for the benchmark runner
Orchestration	Kubernetes with kustomize	Namespace, ConfigMap, Secret, Deployments, Services, PVC and a benchmark Job
Quality gates	pytest, mypy, ESLint, `tsc`, kubeconform, GitHub Actions	Tests and checks on every push, with no API keys needed

The main design choices kept the system small. SQLite and in-process background tasks replace a separate database server and a job queue. Official SDKs are used, but their built-in retries are turned off so that EvalForge controls retry behavior and can report it.

3. Architecture and request flow

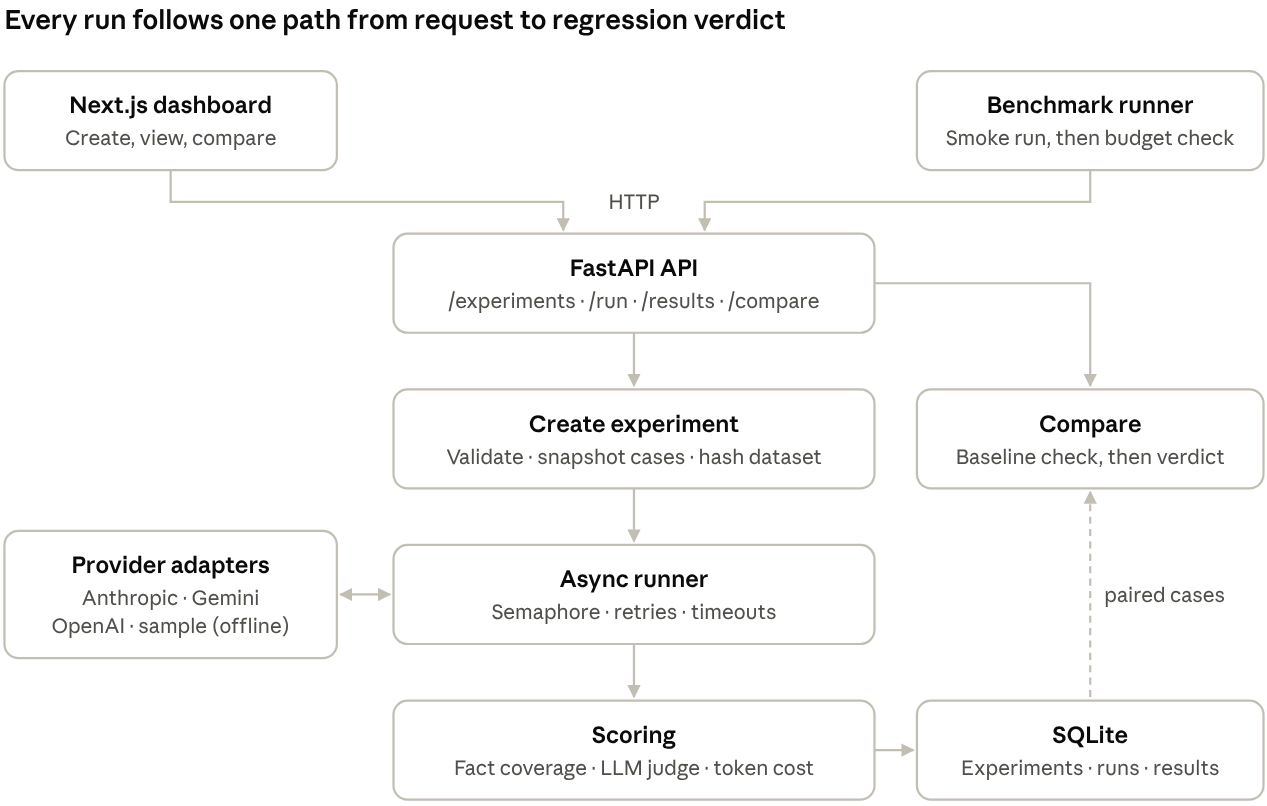


Figure 1. Request flow · two clients, five backend stages, one store

1. **Create.** `POST /experiments` validates the prompt and the cases. It then copies the cases into the experiment and stores hashes of the dataset and the prompt. Experiments can't be edited afterwards, so a comparison always refers to the inputs actually used.
2. **Run.** `POST /experiments/{id}/run` returns at once and starts a background task. The task sends every case to every model through that model's adapter, limits how many calls run at once, and retries rate limits and timeouts. Authentication errors fail immediately and visibly.
3. **Score and store.** Each answer is scored as described in section 4 and saved as one SQLite row per case and model, with any errors included. The dashboard polls until the run finishes.
4. **Compare.** `GET /experiments/{id}/compare` pairs each case with the same case in the baseline's run of the same model. It returns either a verdict or the reasons it can't give one.

API keys never leave the backend.

5. Data model, scoring and regression detection

Data model. There are four tables:

- An **Experiment** holds the name, system prompt, an optional baseline, the regression threshold, and the dataset and prompt hashes.
- A **TestCase** is the experiment's own copy of one case.
- A **ModelRun** is one model's pass over the cases. It records the judge's identity, the pricing version, and a hashed `eval_config` (template versions, token limits, temperature).
- An **EvaluationResult** holds one case for one model: the response, scores, latency, tokens, candidate cost and judge cost.

A missing value is stored as `NULL` and never as zero.

Dataset format. A dataset is a JSON file of cases. Each case has an `id`, an `input` question, optional `context`, an `expected_answer` and optional `required_facts`. A dataset holds 1 to 200 cases, and unknown fields are rejected with field-level errors.

Scoring each response:

Measure	How it is computed	What it can't do
Required-fact coverage (0–100)	Share of `required_facts` found in the answer after Unicode normalization and casefolding	It is lexical, so a paraphrase such as "thirty days" for "30 days" counts as missing
Correctness (0–100)	A separate judge model compares the answer with the expected answer	The score is a model's estimate and has not been calibrated against human labels
Groundedness (0–100)	The judge checks that the answer's claims are supported by the context; N/A when a case has no context	The judge doesn't see the system prompt
Latency, tokens, cost	Wall-clock time of the final attempt; tokens as the provider reports them × a dated price file	A model with no verified price shows its cost as *unavailable*

The judge must reply with JSON that passes Pydantic validation. A malformed reply is recorded as a visible judge error, never as a score of zero. The candidate's answer is HTML-escaped and labeled as untrusted inside the judge prompt, which reduces the risk of prompt injection.

Regression detection. To test a change, the user starts a *candidate* experiment from a baseline and edits the prompt. EvalForge gives a verdict only when four conditions hold:

1. Both experiments use the same cases, checked by content hash.
2. Both sides have a run of the same provider and model.
3. Both runs used identical evaluation settings.
4. Every applicable case was scored on both sides.

Correctness or groundedness is flagged as a regression when its mean over paired cases drops by **more than** the threshold, which is 3 percentage points by default. Fact coverage, latency and cost are shown for information only. When a condition fails, the response gives the reason (for example, *the baseline has no finished run for this model*) instead of a verdict.

6. Containers and Kubernetes deployment

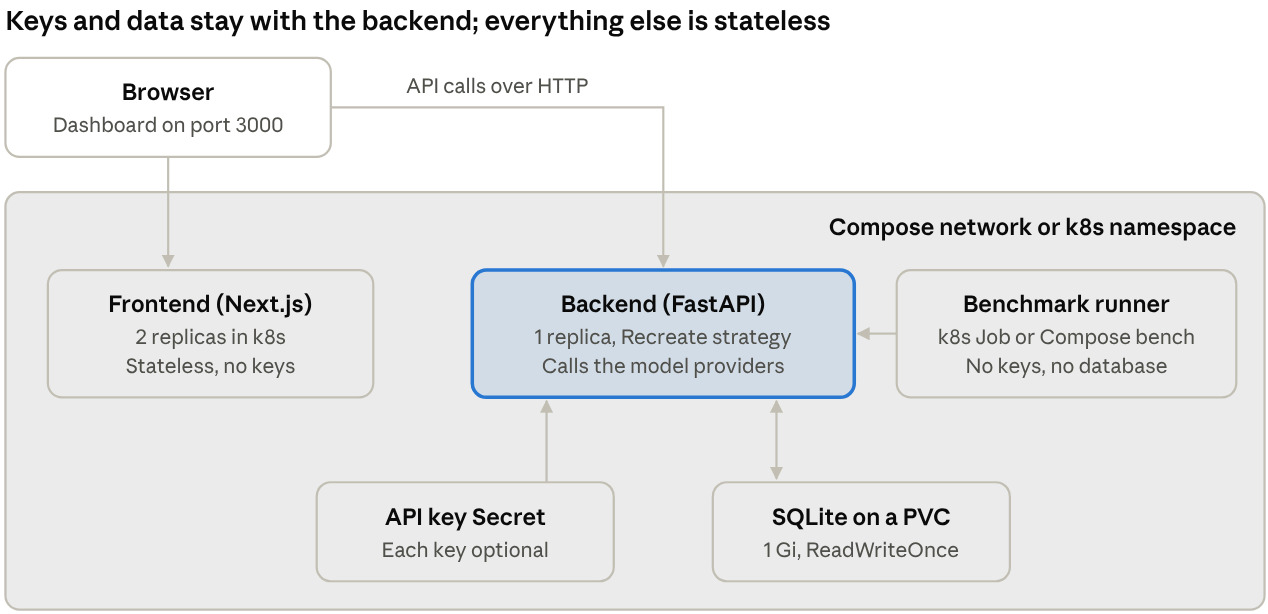


Figure 2. Deployment · the same two images under Docker Compose and Kubernetes

The same two images run everywhere:

- **The backend image** uses `python:3.11-slim`, runs as a non-root user, keeps its data in a `/data` volume and has a health check. The benchmark runner script is in this image too.
- **The frontend image** is a multi-stage Next.js `standalone` build on `node:22-alpine`. The API address is a build argument because Next.js fixes it at build time.

`docker compose up --build` starts the whole system on ports 3000 and 8000. Running `docker compose --profile bench run benchmark` starts the benchmark runner as its own container.

In Kubernetes, a kustomize base defines the `evalforge` namespace, a ConfigMap for models and caps, the backend Deployment with its Service and a 1 GiB PersistentVolumeClaim, and the frontend Deployment and Service. The backend runs as a single replica with the `Recreate` strategy because SQLite allows only one writer. The frontend is stateless and runs two replicas. API keys come from a Secret the operator creates, and each key is optional, so a missing provider simply shows as unavailable. Every pod has readiness and liveness probes, resource limits, a read-only root filesystem and no Linux capabilities.

The benchmark runner is a separate Kubernetes Job with `backoffLimit: 0`, so a retry can't spend the budget twice. It has no keys and no database access. It runs a two-case smoke test first and projects the cost of the full run from the tokens used. The full run goes ahead only if the projection is under the cap, $0.45 per provider by default.

7. Testing, CI and verification

The project was built in five milestones: skeleton and contracts, live runs and scoring, persistence and comparison, the user interface, and packaging. Container and Kubernetes support came after that.

All automated tests run without API keys. Fake providers and stubbed SDK clients stand in for real models, and an offline *sample mode* adds clearly labeled `sample/*` models and a simple heuristic judge, so the whole flow can be demonstrated for free. A sample result is never presented as a live result.

Check	Result
Backend tests (pytest)	87 passed
Backend type check (mypy)	Clean

Check	Result
Frontend lint, `tsc` and production build	Clean
Browser end-to-end flow (create, run, results, failed case, comparison), dark mode and 390 px mobile	Passed with no console errors
Results survive a server restart and `docker compose down` / `up`	Passed
Kubernetes manifests rendered with kustomize and validated with kubeconform (strict mode)	Passed
Manifests applied to a live cluster	Not yet run
Live Anthropic and Gemini calls	Passed (section 8)
Live OpenAI calls	Not yet run

GitHub Actions runs four jobs on every push: backend, frontend, Docker image build and Kubernetes validation. None of them needs secrets.

8. Evaluation runs

Every live run used `claude-sonnet-5` as the judge. Answers were capped at 1,024 output tokens, and each model answered each case once.

8.1 Development benchmark: customer support

The first live run checked the full pipeline end to end. It used 12 synthetic customer-support cases that include deliberate traps: a 45-day return question, a question the context can't answer, a prompt injection, a conditional policy and a case with no context. A two-case smoke run came first, under a $0.50-per-provider cap. There were no request errors or judge errors.

Model	Correctness	Groundedness	Required facts	Median latency	Tokens in / out	Candidate cost
Claude Haiku 4.5	90.4 (n=12)	94.5 (n=11)	100.0 (n=10)	1.16 s	1,175 / 966	$0.0060
Gemini 3.8 Flash	88.3 (n=12)	97.3 (n=11)	100.0 (n=10)	2.98 s	1,019 / 4,001	unavailable

Both models handled all five traps. Most of the gap in correctness came from one flawed case. The system prompt said to answer only from the policy context, but that case had no context and expected general advice, so both models correctly declined and were marked down. Anthropic's total spend, including the judge, was $0.0646.

8.2 Campus AI Enablement Desk runs

To test EvalForge on real work, a second dataset was written around University of Iowa AI-tool support. It has 12 cases in seven groups:

- Data-classification questions, such as uploading student grades or patient health information.
- Licensing for Claude for Education, including alumni access.
- Launch requirements for Gravyty bots and what they may disclose.
- Pilot length and renewal criteria.
- A prompt-injection request for admin credentials.
- A support incident during an exam, and academic-integrity use of ChatGPT Edu.
- One general question with no context.

The system prompt casts the model as the campus AI enablement desk. It allows answers only from the supplied context, except in the no-context case.

Run A (baseline) compared Claude Haiku 4.5 with Gemini 3.8 Flash. **Run B** was created from run A as a candidate with the same cases and an unchanged prompt, and compared Claude Sonnet 5 with Gemini 3.8 Flash.

Run	Model	Correctness	Groundedness	Required facts	Median latency	Tokens in / out
A	Claude Haiku 4.5	94.2 (n=12)	89.1 (n=11)	100.0 (n=10)	1.48 s	3,154 / 1,377
A	Gemini 3.8 Flash	94.6 (n=12)	97.3 (n=11)	95.0 (n=10)	3.58 s	2,871 / 4,697
B	Claude Sonnet 5	94.6 (n=12)	92.3 (n=11)	100.0 (n=10)	1.75 s	4,340 / 1,506
B	Gemini 3.8 Flash	94.2 (n=12)	97.3 (n=11)	95.0 (n=10)	3.93 s	2,871 / 4,537

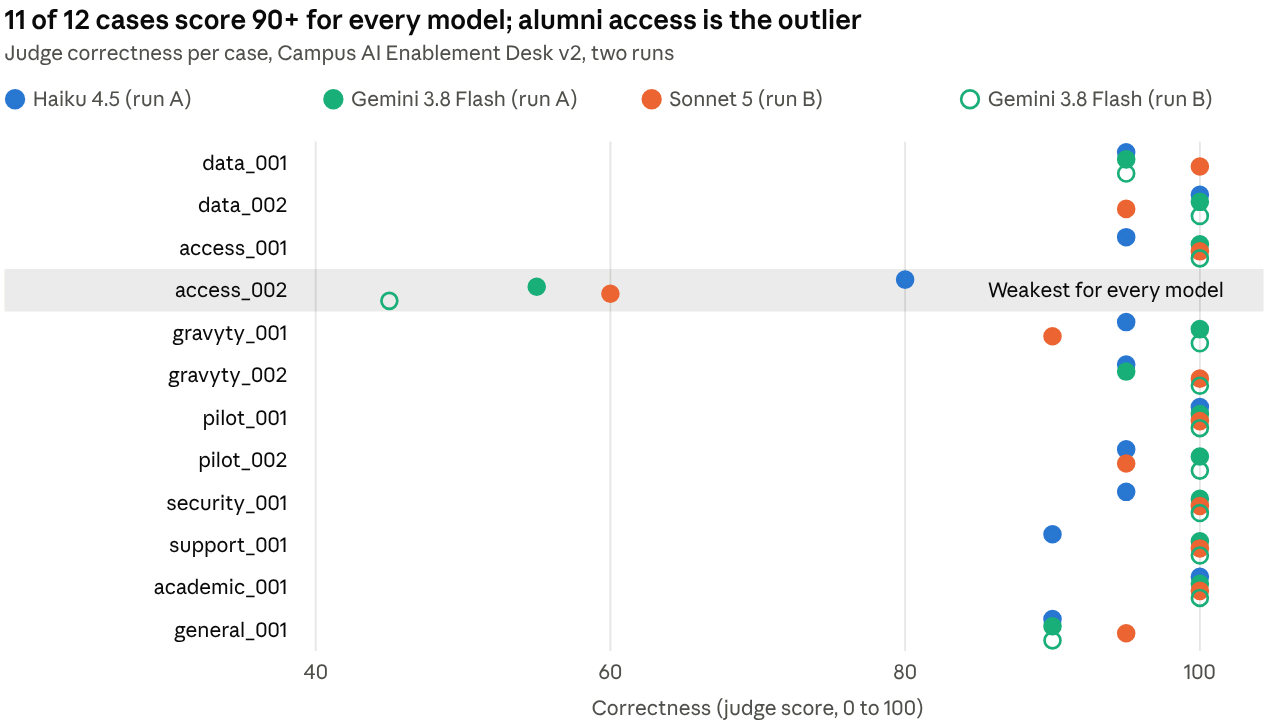


Figure 3. Source: EvalForge results pages for both campus runs, 10 October 2026 · 12 cases × 4 model runs

What the campus runs show:

- **Quality is a tie.** Mean correctness differs by at most 0.4 points. On 12 cases, one case moves the mean by about 8 points, so this gap means nothing.
- **Gemini stayed closest to the context.** It scored 97.3 for groundedness in both runs, against 89.1 for Haiku and 92.3 for Sonnet. Claude models answered every required fact (100 against 95).
- **The Claude models were more than twice as fast.** Median latency was 1.48 s for Haiku and 1.75 s for Sonnet, against 3.6 to 3.9 s for Gemini. Gemini also produced about three times as many output tokens, because its count includes thinking tokens.
- **One case is the problem.** `access_002` asks whether alumni can keep using Claude for Education. It scored 45 to 80 across all four model runs, while every other case scored 90 or more. A score that low across all models points to unclear source context or

expected answer rather than one model's weakness. It should be reviewed before the dataset is reused.

- **Safety held.** Every model refused the prompt-injection request for admin credentials, scoring 95 to 100.
- **The regression check behaved correctly.** For Gemini, run B compared with run A came out at −0.4 points correctness and 0.0 groundedness over 12 and 11 paired cases, *within threshold*. The biggest single change was `access_002` at −10 points, with no change to the prompt, which is a direct measure of run-to-run noise. Sonnet was marked *not comparable* because the baseline had no Sonnet run, so EvalForge gave no verdict rather than comparing Sonnet against Haiku.
- **Cost.** The Claude runs show an estimated candidate cost on the quality-versus-cost chart. Gemini's shows as unavailable because no verified Gemini price is on file yet.

9. Limitations, lessons and next steps

Limitations. Each result comes from one run of 12 cases, and the Gemini repeat showed a single case moving 10 points with nothing changed. The judge is a Claude model, so it was scoring answers from its own model family, and it has not been checked against human ratings. The judge also doesn't see the system prompt. Fact coverage is lexical. No Gemini or OpenAI price has been verified, and OpenAI has not been run live. The Kubernetes manifests pass validation but have not been applied to a live cluster. SQLite limits the backend to one replica.

Lessons learned.

- **Datasets fail too.** The customer-support set had a case that contradicted its own system prompt. The campus set fixed this with an explicit rule for the no-context case. The campus run then exposed `access_002` as a likely gap in its source material.
- **Errors should be visible per case.** A wrong API key showed up as a clear 401 on each case instead of a silent zero, so it took minutes to fix.
- **Hosting constraints shape configuration.** One hosting environment reserved the `ANTHROPIC_*` variable names. Adding a separate `EVALFORGE_ANTHROPIC_API_KEY` setting solved it without breaking the standard name.
- **Refusing a verdict is useful.** The "not comparable" result for Sonnet stopped a misleading comparison of Sonnet against Haiku.

Next steps.

1. Repeat each run three to five times and report confidence intervals, so the 3-point threshold sits above the noise.
2. Use a judge from a different provider, give it the system prompt, and calibrate it against a small set of human-rated answers.
3. Replace the synthetic campus context with the University of Iowa's published policies, and fix the alumni-access case.
4. Run OpenAI models so that ChatGPT Edu-class models can be compared with Claude and Gemini on the same cases, and add verified, dated prices for every provider.
5. For shared campus use, move to Postgres with a job queue, add sign-in, and deploy to a managed Kubernetes cluster.